Rapport

Autocomplétion et structures de données

Structures de données — IFT2015

Par

David Serafini
Damia Boudjema

Présenté à

François Major

7 avril 2026

Université de Montréal

Répondez aux quatre questions ci-dessous.
Les réponses doivent utiliser une ***notation asymptotique précise (Big-O)***
et être accompagnées d'une justification claire.

Q1 — Invariants des structures de données (12 pts)

Pour chacune des trois structures de données utilisées dans ce projet, définissez les **invariants structurels** (propriétés qui doivent toujours être vraies) et expliquez comment les opérations principales maintiennent ces invariants.

1a. Table de fréquences (`HashMap`) (4 pts)

La `HashFrequencyTable` repose sur deux champs : une `HashMap<String, Integer>` qui associe chaque token à son nombre d'occurrences, et un entier `totalCount` qui représente le total cumulé de toutes les observations.

Invariants à maintenir :

1. Tout token présent dans la map a un compteur ≥ 1. On ne stocke jamais la valeur 0. si un token n'a pas encore été observé, il est absent de la map.
2. `totalCount` vaut toujours la somme de tous les compteurs stockés : `totalCount` $= \sum_{t \in \texttt{map}} \texttt{map.get}(t)$.
3. `vocabulary()` retourne exactement `map.keySet()`, soit l'ensemble des tokens ayant été vus au moins une fois.

`increment(t)` utilise `getOrDefault(t, 0) + 1` : un token absent reçoit directement 1 jamais 0, donc l'invariant 1 est toujours respecté. `totalCount` est incrémenté en même temps, ce qui maintient l'invariant 2. `incrementBy(t, c)` suit la même logique avec $+c$ au lieu de $+1$, comme $c \geq 1$, aucun problème. Les méthodes `get` et `total` sont de simples lectures, elles ne changent rien.

Toutes les opérations s'exécutent en $\mathcal{O}(1)$ amorti grâce au hachage.

1b. Arbre de préfixes (`PrefixTrie`) (4 pts)

La `PrefixTrie` est une structure arborescente où chaque noeud (`TrieNode`) contient une `HashMap<Character, TrieNode>` pour ses enfants, un booléen `isEndOfWord` et un entier `frequency`. La trie suit aussi `size` (nombre de mots distincts insérés) et `nodeCount` (noeuds créés hors racine).

Invariants :

1. Chaque chemin de la racine vers un noeud représente un préfixe unique. Deux mots

qui commencent pareil ("the" et "there") partagent les mêmes noeuds pour les caractères communs et c'est précisément ce qui fait l'intérêt du trie.

2. Un noeud a `isEndOfWord = true` ssi le chemin jusqu'à lui forme un mot inséré. Dans ce cas, `frequency` ≥ 1.
3. `size` est égal au nombre de noeuds avec `isEndOfWord = true` dans l'arbre entier.

`insert(word, freq)` descend caractère par caractère depuis la racine en créant au passage les noeuds manquants (`nodeCount++`). Au noeud final si c'est une nouvelle fin de mot `isEndOfWord` passe à `true` et `size` est incrémenté. `frequency` est toujours augmenté de `freq` donc il reste ≥ 1 après la première insertion. Les opérations `search` et `complete` ne font que lire la structure aucun invariant n'est affecté.

La descente dans le trie se fait en $\mathcal{O}(|w|)$ où $|w|$ est la longueur du mot.

1c. Tas min (`PriorityQueue`) (4 pts)

Dans `HeapTopKStrategy` on utilise une `PriorityQueue` Java avec un comparateur par fréquence croissante (et lexicographique décroissant à égalité). L'idée est que le sommet du tas est toujours le "moins bon" parmi les k candidats retenus ce qui permet de décider rapidement si un nouveau token vaut la peine d'entrer.

Invariants :

1. **Propriété de tas min :** Chaque noeud est plus petit que ses enfants selon le comparateur. Le minimum se trouve donc toujours à la racine, accessible en $\mathcal{O}(1)$.
2. **Taille $\leq k$:** Dès qu'on dépasse k éléments on retire le minimum. Le tas ne contient jamais plus de k tokens simultanément.
3. **Contenu optimal :** En fin de parcours le tas contient les k meilleurs tokens vus. Comme on n'élimine que le minimum aucun token plus fréquent ne peut avoir été rejeté à tort.

`heap.add(token)` insère l'élément et le fait remonter (*sift-up*) pour rétablir l'ordre en $\mathcal{O}(\log k)$. `heap.poll()` si la taille dépasse k retire le minimum et fait redescendre le remplaçant (*sift-down*) en $\mathcal{O}(\log k)$. Le `Collections.reverse()` final sert juste à présenter les résultats en ordre décroissant il ne modifie pas les invariants du tas.

La complexité globale de `topK` est $\mathcal{O}(V \log k)$ pour un vocabulaire de taille V.

Q2 — Complexité asymptotique de l'entraînement (12 pts)

Considérez un corpus contenant N tokens. Le vocabulaire a une taille V ($V \leq N$). La longueur moyenne des tokens est supposée constante.

Analysez la complexité de la méthode `train(List<List<String» sentences)` en décomposant chaque étape.

Variables à utiliser : N (tokens), V (vocabulaire).

2a. Construction de la table de fréquences (`unigrams`) (4 pts)

La construction de la table de fréquence se fait avec la fonction increment(w). Cette fonction utilise trois opérations : 1. put(token, map.getOrDefault(token, 0) + 1), sois un put() sur une HashMap : $\mathcal{O}(lettres)$ où lettres représente le nombre de lettres dans un mot. Dans ce put, une autre opération est appelée. 2. getOrDefault(token, 0) sois un get() sur une HashMap avec une clé en String : $\mathcal{O}(lettres)$ 3. totalCount++ : $\mathcal{O}(1)$. En regroupant les complexités de la fonction, on obtient $\mathcal{O}(lettres)$ + $\mathcal{O}(lettres)$ + O(1) = $\mathcal{O}(2xlettres + 1) = \mathcal{O}(lettres)$.

La fonction insert(String word, int frequency) est appelée pour chaque mot dans le corpus, donc N fois. Sa complexité est donc $\mathcal{O}(Nxlettres)$. Puisque lettres est bornée par une constante, la longueur du mot le plus long, la complexité peut être réduite à $\mathcal{O}(N)$.

2b. Construction du trie (4 pts)

La construction du trie se fait avec l'opération trie.insert(w, 1). L'insertion d'un mot dans un trie est de l'ordre de la taille du mot à insérer, puisque la fonction insert(String word, int frequency) boucle sur la taille du mot. On appel donc word.toCharArray() pour initialiser la boucle, une opération qui coûte $\mathcal{O}(lettres)$ où lettres représente le nombre de lettres dans un mot. Dans cette boucle,les opérations appelées sont les suivantes : 1. get(c) sur une HashMap : $\mathcal{O}(1)$ amorti 2. put(c, NewTrieNode()) sur une Hash map : $\mathcal{O}(1)$ 3. nodeCount++ : $\mathcal{O}(1)$ 4.get(c) sur une HashMap (encore) : $\mathcal{O}(1)$.

Toutes ces opérations sont en $\mathcal{O}(1)$. Puisque la boucle s'exécute lettres fois, elle est de $\mathcal{O}(lettres)$. Les opérations exécutées après la boucle sont tous en $\mathcal{O}(1)$. En regroupant les complexités de la fonction, on obtient $\mathcal{O}(lettres)$ + $\mathcal{O}(lettres)$ + $\mathcal{O}(1)$ = $\mathcal{O}(2xlettres+1)$ = $\mathcal{O}(lettres)$.

La fonction insert(String word, int frequency) est appelée pour chaque mot dans le corpus, donc N fois. Sa complexité est donc $\mathcal{O}(Nxlettres)$. Par le même raisonnement qu'à la question précédente, $\mathcal{O}(Nxlettres)$ devient $\mathcal{O}(N)$.

En terme d'espace mémoire, on ne crée de nouveaux node dans le trie que pour des nouveaux mots. La complexité mémoire est donc de $\mathcal{O}(Vxlettres) = \mathcal{O}(V)$.

2c. Construction des tables bigrammes et trigrammes (4 pts)

La construction des tables bigrammes commence avec un get sur un HashMap en $\mathcal{O}(1)$ amorti. Ensuite, computeIfAbsent(w$_p rev, k- > newHashFrequencyTable())estappel.SoitonaccdeuneH$

Au total, les bigrammes sont gérés N fois en $\mathcal{O}(lettres)$, ce qui donne une vitesse de $\mathcal{O}(N)$. En complexité mémoire, puisqu'il s'agit d'une HashMap de HashMap, on remarque qu'on store jusqu'à V valeur qui, chacune, store jusqu'à V entrées. Au total, on trouve une complexité mémoire de $\mathcal{O}(V^2)$.

Pour les trigrammes, la seule différence est que la clé est une paire de strings. Il faut donc rajouter une opération trivial de concaténation de string $\mathcal{O}(lettres)$. computeUfAbsent(,)$estpresqueident$ lettres)=O(lettres)$.increment(w)estinchang$.O(lettres).

La complexité total, par le même raisonnement qu'avec les bigrammes, est $\mathcal{O}(N)$. En terme de mémoire, le raisonnement est le même qu'avec les bigrammes, mais avec des triplets au lieu de paires. La complexité mémoire est donc $\mathcal{O}(V^3)$.

Q3 — Complexité asymptotique des requêtes (10 pts)

Considérez une requête depuis un modèle déjà entraîné sur un vocabulaire de taille V. On demande les k meilleures suggestions pour un contexte donné. Le préfixe soumis à `complete` a une longueur L.

Variables à utiliser : V, k, L.

3a. `topK` via la table de fréquences (4 pts)

Pour trouver les k tokens les plus fréquents, on passe en revue tout le vocabulaire (V tokens) et on maintient un tas min de taille $\leq k$.

Pour chaque token, on lit sa fréquence dans la `HashMap` en $\mathcal{O}(1)$ amorti, on l'insère dans le tas en $\mathcal{O}(\log k)$ et si le tas dépasse k éléments on retire le minimum en $\mathcal{O}(\log k)$. En répétant ça pour les V tokens, on obtient :

$$\mathcal{O}(V \log k)$$

Le tri final de la liste résultante coûte $\mathcal{O}(k \log k)$, ce qui est négligeable puisque $k \leq V$.

3b. `complete` via le trie (6 pts)

Exprimer la complexité en distinguant la phase de descente au préfixe et la phase de DFS + sélection top-k.

La méthode `complete(prefix, k)` se divise en deux phases bien séparées.

Phase 1 - Descente au noeud préfixe : On suit les L caractères du préfixe depuis la racine un par un. L'accès dans la `HashMap` des enfants de chaque noeud coûte $\mathcal{O}(1)$ amorti. Si un caractère est absent à n'importe quelle étape on s'arrête immédiatement. Coût total : $\mathcal{O}(L)$.

Phase 2 - DFS + sélection top-k : Depuis le noeud préfixe, on fait un parcours en profondeur du sous-trie. Soit P le nombre de noeuds visités et V' le nombre de mots trouvés sous ce préfixe ($V' \leq V$). Pour chaque mot on fait une insertion dans le tas en $\mathcal{O}(\log k)$. Cette phase coûte donc $\mathcal{O}(P + V' \log k)$.

Complexité totale :

$$\mathcal{O}(L + P + V' \log k)$$

Dans le pire cas (préfixe très court ou vide), P atteint le nombre total de nœuds du trie et $V' = V$. En pratique un préfixe plus long réduit fortement P et V' ce qui rend `complete` bien plus efficace que `topK` pour des requêtes précises.

Q4 — Comparaison d'implémentations alternatives (6 pts)

Choisissez **une** des structures de données utilisées dans ce projet et comparez-la à une alternative plausible.

Exemples : trie vs tableau trié, tas min vs tri complet,
table de hachage vs arbre binaire de recherche équilibré.

Votre réponse doit inclure :

- une description des deux approches,
- la complexité des opérations principales pour chacune,
- une discussion sur l'utilisation mémoire,
- une conclusion sur le contexte où chaque approche serait préférable.

L'autocompléteur utilise un Trie. Tel que mentionné dans la section 1, chaque noeud du Trie contient une HashMap<Character, TrieNode> pour ses enfants, un booléen `isEndOfWord` et un entier `frequency`. La trie suit aussi `size` (nombre de mots distincts insérés) et `nodeCount` (noeuds créés hors racine). Parcourir l'arbre est une opération en O(|w|), soit la longueur des mots. La complexité mémoire d'un Trie, tel qu'évalué dans la section 2, est O(V), où V est le nombre de mots dans le vocabulaire.

L'alternative proposé est le tableau trié. Le tableau serait trié par ordre alphabétique et serait parcourus par recherche binaire. Chaque comparaison de la recherche binaire se ferait en O(|w|), car nous comparons deux strings lettres par lettres. Une recherche binaire sur n éléments est de complexité O(logn). Puisque le tableau contient V entrées, il aurait une complexité en temps de O(logV). En terme de complexité mémoire, le tableau serait de taille O(V), soit une entrée pour chaque mot du vocabulaire.

Le Trie est préférable pour le contexte d'autocompléteur grâce à sa rapidité d'opérations. Il n'y a pas d'avantage remarquable au tableau trié ici, à moins que les échantillons traités soit si petits qu'un Trie soit simplement plus complex à implémenter. Autre que ce cas presque trivial, Le Trie est supérieur en tout points.

Bibliographie

Les références doivent être présentées selon le style **APA**.

Exemple :

— Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Java* (6e éd.). Wiley.

Ajoutez ci-dessous toutes les sources utilisées :

—

—

—

—